

MacroSnaps: Architecture Overview

Last updated March 12, 2026

The One-Sentence Version

`data.json` holds everything. `build.py` assembles it into `macrosnaps-globe.html`. A set of scripts keeps `data.json` fresh. Git pushes the output to GitHub Pages.

Helicopter View

File / Script	Type	Role	Run when
<code>data.json</code>	Source data	Single source of truth for all content and data	Edited by scripts, never by hand
<code>macrosnaps-shell.html</code>	Source UI	App shell with all JS, CSS, layout (no data)	Edit for UI changes only
<code>build.py</code>	Build script	Injects data into shell, validates, writes output	After every data or content change
<code>macrosnaps-globe.html</code>	Build output	The live app. Self-contained HTML. Never edit directly	Generated by <code>build.py</code>
<code>sync_sheet.py</code>	Data script	Pulls macro forecasts from Google Sheet into <code>data.json</code>	When consensus forecasts change
<code>fetch_market_data.py</code>	Data script	Pulls live market prices and commodity data from Yahoo/FRED	Daily
<code>update_commodity_stories.py</code>	Story script	Rewrites commodity stories for any commodity that moved past threshold	Daily (after <code>fetch_market_data.py</code>)
<code>update_stories.py</code>	Story script	Rewrites per-metric stories for any metric that moved past threshold	Daily (after <code>fetch_market_data.py</code>)
<code>update_headlines.py</code>	Story script	Drafts country and global headline stories via Claude API + web search	Daily
<code>refetch_historical.py</code>	Maintenance script	Restores frozen historical chart arrays from FRED / Yahoo Finance	Manually, when charts need repair
<code>headline_review.html</code>	Browser tool	Review, edit, and approve headline drafts before applying	After each <code>update_headlines.py</code> draft run
<code>.env</code>	Config	Holds ANTHROPIC_API_KEY and FRED_API_KEY. Never commit to git	One-time setup

The Four Source Files

These are the only files you ever edit or that scripts write to. Everything else is either generated or supporting.

`data.json`

The entire content layer of the app. Contains:

- 12 country blocks, each with macro metrics, market metrics, stories, metricBriefs, fxRegime, and frozen historical chart arrays
- 9 commodity items with prices, spark arrays, and stories
- 3 global story slots (beginner, moderate, expert)

Nothing is displayed to users that does not live in this file. It is the sole input to `build.py`.

`macrosnaps-shell.html`

The app shell. Contains all JavaScript, CSS, and HTML layout. Has no data of its own. At build time, `build.py` injects `window.__MACROSNAPS_DATA__` from `data.json` into the shell just before `</head>` and writes the result to `macrosnaps-globe.html`.

Edit this file for any UI, layout, or JS logic changes.

`build.py`

The assembly script. On every run it:

1. Validates `data.json` (structure, required fields, story completeness)
2. Injects the data blob into the shell
3. Writes `macrosnaps-globe.html`
4. Saves a dated backup to `backups/` (pruned after 30 days)
5. Auto-commits after a successful build

Run with `--validate-only` to check `data.json` without writing any files.

`macrosnaps-globe.html`

The live app. Generated by `build.py`. Self-contained: all data is inlined. Hosted on GitHub Pages at <https://ralphlazar.github.io/macrosnaps/macrosnaps-globe.html>. Never edit this file directly.

The Scripts

`sync\_sheet.py`

Pulls the 6 macro forecast metrics (GDP, CPI, Unemployment, Budget Deficit, Current Account, Policy Rate) from Ralph's Google Sheet (published CSV) and writes them into `data.json`.

- Run without flags first: shows a preview of what will change. Nothing is written.
- Run with `--apply` to write changes.
- Trigger: when consensus forecasts change in the sheet (infrequent, a few times per month at most).
- Does not touch stories, market metrics, historical arrays, or any other field.

`fetch\_market\_data.py`

Pulls all live market data and writes it into `data.json`. Runs daily.

Fetches:

- Stock Market YTD, 10Y Bond Yield, Yield Curve, Equity Vol, Corp Spread, Sov CDS, FX pair, FX Vol for all 12 countries (from Yahoo Finance and FRED)
- Price, YoY change, spark array, and asOf for all 9 commodities (from Yahoo Finance continuous futures tickers)

Known permanent gaps (not bugs): RUS stock/vol (IMOEX delisted), CHN and BRA bond/curve/CDS (no FRED series), 4 data-void metrics (Equity Vol, Corp Spread, Sov CDS, FX Vol) displayed as "not available."

Run `--dry-run` first on a new machine or after a long gap.

`update\_commodity\_stories.py`

Checks each commodity's current price against `storyWrittenAtPrice` stored in `data.json`. Rewrites stories at all three levels (beginner, moderate, expert) for any commodity that has moved past its threshold. Applies directly to `data.json` with no review step.

- Thresholds: 10% for Natural Gas, 5% for everything else.

- On quiet days exits in under a second with no API call.
- Requires ANTHROPIC_API_KEY in .env.
- Added March 12, 2026. All 9 commodities bootstrapped on first run.

`update\_stories.py`

Diff's data.json against the last git commit. Identifies metrics whose value field has changed past a configurable threshold. Calls the Claude API (Sonnet) and rewrites stories for affected metrics at all three levels in one pass.

- Run after both sync_sheet.py --apply and fetch_market_data.py to catch all changes.
- Does not touch commodity stories (handled by update_commodity_stories.py).
- Requires ANTHROPIC_API_KEY in .env.
- Relies on consistent git commits to correctly target drift. Always commit after a successful build.

`update\_headlines.py`

Drafts country-level and global headline stories via the Claude API. Two modes:

Draft mode (no flags): runs the full pipeline and saves a draft file.

1. Sonnet + web search: harvests recent macro data for all 12 countries in one call (recent CPI prints, GDP reads, central bank decisions).
2. Haiku x3 batches: writes 3 bullets per level (beginner, moderate, expert) for each country, with recent data leading and forecast values as background context.
3. Sonnet + web search: writes 3 global story cards at all levels (three-act arc: Today's Story, Biggest Movers, The Connection).

Saves to stories_draft_YYYY-MM-DD.json. Does not touch data.json.

Apply mode (--apply): reads an approved file and writes stories into data.json.

```
python3 update_headlines.py --apply stories_approved_YYYY-MM-DD.json
```

Review happens in headline_review.html (browser tool) between draft and apply.

`refetch\_historical.py`

Pulls historical data from FRED and Yahoo Finance and writes _frozen_historical arrays into data.json. Run manually when charts need repair or population.

Warning: overwrites Current Account arrays with raw dollar values, undoing the % of GDP conversion. Re-apply CA conversion after any refetch run.

Key settings: FORCE_OVERWRITE (default False), MIN_POINTS (default 5).

Data Flow Diagram



```
      |
      v
    git push
      |
      v
  GitHub Pages (live)
```

The Daily Bash Ritual (in order)

```
cd ~/Downloads/macrosnaps

# 1. Preview sheet sync (read only, nothing written)
python3 sync_sheet.py

# 2. Apply sheet sync if changes look correct
python3 sync_sheet.py --apply

# 3. Fetch live market data and commodity prices
python3 fetch_market_data.py

# 4. Rewrite commodity stories if prices moved
python3 update_commodity_stories.py

# 5. Rewrite per-metric stories if values moved
python3 update_stories.py

# 6. Draft country and global headlines
python3 update_headlines.py
# Open headline_review.html, review, approve, export stories_approved_YYYY-MM-DD.json
python3 update_headlines.py --apply stories_approved_YYYY-MM-DD.json

# 7. Build
python3 build.py

# 8. Commit and push
git add -A && git commit -m "Daily update $(date +%Y-%m-%d)"
git push origin master

# 9. Verify live site (wait ~60 seconds first)
# https://ralphlazar.github.io/macrosnaps/macrosnaps-globe.html
```

Steps 1-2 can be skipped on days when no forecast values have changed in the sheet. Steps 4 and 5 will exit quickly with no API call if nothing has moved past threshold.

Key Conventions

Never edit `macrosnaps-globe.html` **directly**. It is overwritten on every build.

Never manually edit `_frozen_historical` **or** `_frozen_weatherGrid` **arrays**. Use `refetch_historical.py` instead.

Always commit after a successful build. `update_stories.py` uses `git diff` to detect what has changed since the last commit. Skipping a commit means drift goes undetected.

Validate before editing `data.json` **by hand**. Run `python3 build.py --validate-only` first. If the build fails after an edit, do not push.

Never paste API keys into chat or commit them to git. Always use `.env` (gitignored). The Anthropic key was exposed twice in early sessions and has been rotated. Use `nano .env` or interactive input to set secrets.

`data.json` **is the only file that changes every day**. The shell and build script are stable. Edit them only for UI or build logic changes, not for content.

Environment Setup (one-time)

```
# Python packages  
pip3 install anthropic python-dotenv requests yfinance
```

```
# .env file (gitignored)  
# ~/Downloads/macrosnaps/.env contains:  
ANTHROPIC_API_KEY=your_key_here  
FRED_API_KEY=your_key_here
```

FRED API key is free at https://fred.stlouisfed.org/docs/api/api_key.html.

Hosting

- **Repo:** <https://github.com/ralphlazar/macrosnaps> (branch: master, root folder)
- **Live URL:** <https://ralphlazar.github.io/macrosnaps/macrosnaps-globe.html>
- **Deploy:** automatic on push to master. No build step needed server-side (output is self-contained HTML).
- **Local preview:** open -a Safari ~/Downloads/macrosnaps/macrosnaps-globe.html or python3 -m http.server 8000 then <http://localhost:8000/macrosnaps-globe.html>